

# wq-sample-label-datasheet-templates

## Summary

The purpose of this R markdown is to ...

Some packages that might be of use:

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.3
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(stringr)
```

```
library(kableExtra)
```

```
##
```

```
## Attaching package: 'kableExtra'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      group_rows
```

The dplyr package is specifically designed to do data manipulation in a way that's relatively easy to read using things called pipes (%>%). See the chapter in this textbook here for some examples of using dplyr.

## 1. User Inputs

Put info from excel FILLME here as variables. Since we updated the methods we'll probably need to add:

- random\_start for the starting random number
- samp\_type (GB,IS, SS, etc.)

Variable names should be lowercase with no spaces (use underscores instead) and not too long (abbreviate where it makes sense).

```
project <- "WWS" #the large project code
```

```
study <- "TEST" #the study code
```

## 2. Bottle Calculator

Based on number of sites and samples per site, calculate bottle types needed as data.frame, then we can print nicely.

## 3. Label IDs

We'll use this to generate sample IDs and random IDs for labels. For a start just use R functions to take the info from 1 and convert to the IDs. I think you have the most updated things we need but I think we need:

- **SampleID:** project\_site\_type\_date\_time
- **RandomID:** project\_r###
- **ISCOID:** just the number order it goes in the isco?

```
#paste function works well to join things together  
paste0("A", "B", "C")
```

```
## [1] "ABC"
```

```
paste("A", "B", "C")
```

```
## [1] "A B C"
```

```
paste("A", "B", "C", sep="_")
```

```
## [1] "A_B_C"
```

```
#the seq function is good for generating a sequence  
seq(1,4)
```

```
## [1] 1 2 3 4
```

```
seq(as.Date("2023-02-01"), as.Date("2024-01-03"), by="month")
```

```
## [1] "2023-02-01" "2023-03-01" "2023-04-01" "2023-05-01" "2023-06-01"
```

```
## [6] "2023-07-01" "2023-08-01" "2023-09-01" "2023-10-01" "2023-11-01"
```

```
## [11] "2023-12-01" "2024-01-01"
```

```
#we can store all this in a data.frame similar to excel
```

```
data <- data.frame(project = project,  
                   study=study,  
                   dates = seq(as.Date("2023-02-01"),  
                               as.Date("2024-01-03"), by="month"))
```

```
data
```

```
##   project study      dates  
## 1     WWS  TEST 2023-02-01  
## 2     WWS  TEST 2023-03-01  
## 3     WWS  TEST 2023-04-01  
## 4     WWS  TEST 2023-05-01  
## 5     WWS  TEST 2023-06-01  
## 6     WWS  TEST 2023-07-01  
## 7     WWS  TEST 2023-08-01  
## 8     WWS  TEST 2023-09-01  
## 9     WWS  TEST 2023-10-01  
## 10    WWS  TEST 2023-11-01  
## 11    WWS  TEST 2023-12-01
```

```
## 12      WWS  TEST 2024-01-01
```

## 4. Specifying analyte

We might need to do some experimenting here. For a set of ISCO samples or a small number of grab samples I think we could just let the user specify the analytes they plan to run (i.e., `analyte <- c("DOC", "DOM")`). If we want to get fancier and let people run only a subset of samples for something we'll need to get fancier. But for now maybe we just allow all or nothing.

## 5. Generating the .csv for labels

We'll want to write the labels we need to a .csv file so we can use it for making labels. We'll want to just pull the columns we want.

```
print_data <- data %>% select(project, study)
print_data
```

```
##      project study
## 1      WWS  TEST
## 2      WWS  TEST
## 3      WWS  TEST
## 4      WWS  TEST
## 5      WWS  TEST
## 6      WWS  TEST
## 7      WWS  TEST
## 8      WWS  TEST
## 9      WWS  TEST
## 10     WWS  TEST
## 11     WWS  TEST
## 12     WWS  TEST
```

```
#write.csv(print_data, "filename.csv", row.names=FALSE)
```

## 6. Making the tables

This part will be a little more complicated, since we want specific formatting. I think if we use the LaTeX command below (which is what the Rmarkdown uses to convert to PDF), it should start a new page that we can format how we like.

```
\newpage
```

Date: \_\_\_\_\_

Technician: \_\_\_\_\_

Project Code: \_\_\_\_\_

Study Code: \_\_\_\_\_

```
kable(data, booktabs = TRUE) %>%  
  kable_styling(latex_options = c("striped", "hold_position"),  
    full_width = FALSE,  
    font_size = 10) %>%  
  row_spec(0, bold = TRUE, color = "white", background = "#D7261E")
```

| project | study | dates      |
|---------|-------|------------|
| WWS     | TEST  | 2023-02-01 |
| WWS     | TEST  | 2023-03-01 |
| WWS     | TEST  | 2023-04-01 |
| WWS     | TEST  | 2023-05-01 |
| WWS     | TEST  | 2023-06-01 |
| WWS     | TEST  | 2023-07-01 |
| WWS     | TEST  | 2023-08-01 |
| WWS     | TEST  | 2023-09-01 |
| WWS     | TEST  | 2023-10-01 |
| WWS     | TEST  | 2023-11-01 |
| WWS     | TEST  | 2023-12-01 |
| WWS     | TEST  | 2024-01-01 |